

Data Preprocessing

```
In [2]: # importing Lib
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [4]: data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany', 'France', 'Spain', 'Germany', 'France', 'Spain'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000, 79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)
print(df)
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
In [5]: # checking basic information
print(df.info())
print('-----')
print(df.isnull()) #checking null value
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Country     10 non-null    object
1   Age         9 non-null     float64
2   Salary      9 non-null     float64
3   Purchased   10 non-null    object
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes
None
```

```
-----
      Country   Age  Salary  Purchased
0      False  False   False      False
1      False  False   False      False
2      False  False   False      False
3      False  False   False      False
4      False  False   True       False
5      False  False   False      False
6      False  True    False      False
7      False  False   False      False
8      False  False   False      False
9      False  False   False      False
```

```
In [6]: # taking raw or testing data set
X_raw = df.iloc[:, :-1].values
y_raw = df.iloc[:, -1].values

#Pandas
dfx_raw = df.iloc[:, :-1]
dfy_raw = df.iloc[:, -1]

print(f"val are X raw: {X_raw} \n Y_raw: {y_raw} \n Panad X: {dfx_raw} \n pandas Y \n{
```

```

val are X raw: [['France' 44.0 72000.0]
['Spain' 27.0 48000.0]
['Germany' 30.0 54000.0]
['Spain' 38.0 61000.0]
['Germany' 40.0 nan]
['France' 35.0 58000.0]
['Spain' nan 52000.0]
['France' 48.0 79000.0]
['Germany' 50.0 83000.0]
['France' 37.0 67000.0]]
Y_raw: ['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
Panad X:      Country   Age   Salary
0   France  44.0  72000.0
1   Spain  27.0  48000.0
2   Germany 30.0  54000.0
3   Spain  38.0  61000.0
4   Germany 40.0      NaN
5   France 35.0  58000.0
6   Spain  NaN  52000.0
7   France 48.0  79000.0
8   Germany 50.0  83000.0
9   France 37.0  67000.0
pandas Y
0      No
1      Yes
2      No
3      No
4      Yes
5      Yes
6      No
7      Yes
8      No
9      Yes
Name: Purchased, dtype: object

```

```

In [7]: # Copy the variable copy()
X_raw_copy = X_raw.copy()
y_raw_copy = y_raw.copy()

X_raw_copy

```

```

Out[7]: array(['France', 44.0, 72000.0],
              ['Spain', 27.0, 48000.0],
              ['Germany', 30.0, 54000.0],
              ['Spain', 38.0, 61000.0],
              ['Germany', 40.0, nan],
              ['France', 35.0, 58000.0],
              ['Spain', nan, 52000.0],
              ['France', 48.0, 79000.0],
              ['Germany', 50.0, 83000.0],
              ['France', 37.0, 67000.0]], dtype=object)

```

Medeling with missing Data

```

In [8]: # cecking with first info()
df.info()

```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Country     10 non-null    object
1   Age         9 non-null     float64
2   Salary      9 non-null     float64
3   Purchased   10 non-null    object
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes
```

```
In [9]: # checcking for numpy array info is not a class of numpy module
X_raw_copy.info()
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_10848\3194039848.py in <module>
      1 # checcking for numpy array info is not a class of numpy module
----> 2 X_raw_copy.info()

AttributeError: 'numpy.ndarray' object has no attribute 'info'
```

```
In [10]: X_raw_copy = pd.DataFrame(X_raw_copy)
X_raw_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   0        10 non-null    object
1   1         9 non-null    object
2   2         9 non-null    object
dtypes: object(3)
memory usage: 368.0+ bytes
```

```
In [11]: X_raw_copy = X_raw_copy.values
```

```
In [12]: X_raw_copy
```

```
Out[12]: array([[ 'France', 44.0, 72000.0],
      ['Spain', 27.0, 48000.0],
      ['Germany', 30.0, 54000.0],
      ['Spain', 38.0, 61000.0],
      ['Germany', 40.0, nan],
      ['France', 35.0, 58000.0],
      ['Spain', nan, 52000.0],
      ['France', 48.0, 79000.0],
      ['Germany', 50.0, 83000.0],
      ['France', 37.0, 67000.0]], dtype=object)
```

```
In [13]: type(X_raw_copy)
```

```
Out[13]: numpy.ndarray
```

```
In [14]: dfx_raw.isnull().sum()
```

```
Out[14]: Country    0
         Age       1
         Salary    1
         dtype: int64
```

```
In [15]: print(X_raw_copy.isnull().sum)
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_10848\3054524884.py in <module>
----> 1 print(X_raw_copy.isnull().sum)

AttributeError: 'numpy.ndarray' object has no attribute 'isnull'
```

```
In [16]: dfx_raw = dfx_raw.fillna(0) # change nul to 0
         dfx_raw
```

```
Out[16]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	0.0
5	France	35.0	58000.0
6	Spain	0.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [17]: dfx_raw = dfx_raw.replace(0,np.nan) #replace 0 with null np.nan
         dfx_raw
```

```
Out[17]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [18]: # extracting the index
index_no = dfx_raw.columns.get_loc('Age')
index_no
```

```
Out[18]: 1
```

```
In [19]: dfx_raw.iloc[:, 1] = dfx_raw.iloc[:, 1].fillna(5)
dfx_raw
```

```
Out[19]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	5.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [20]: dfx_raw = df.iloc[:, :-1]
dfx_raw
```

```
Out[20]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

Missing value manipulation

```
In [21]: dfx_raw
```

Out[21]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

In [22]: !pip list

Package	Version

alabaster	0.7.12
anaconda-client	1.11.0
anaconda-navigator	2.3.1
anaconda-project	0.11.1
anyio	3.5.0
appdirs	1.4.4
argon2-cffi	21.3.0
argon2-cffi-bindings	21.2.0
arrow	1.2.2
astroid	2.11.7
astropy	5.1
atomicwrites	1.4.0
attrs	21.4.0
Automat	20.2.0
autopep8	1.6.0
Babel	2.9.1
backcall	0.2.0
backports.functools-lru-cache	1.6.4
backports.tempfile	1.0
backports.weakref	1.0.post1
bcrypt	3.2.0
beautifulsoup4	4.11.1
binaryornot	0.4.4
bitarray	2.5.1
bkcharts	0.2
black	22.6.0
bleach	4.1.0
bokeh	2.4.3
boto3	1.24.28
botocore	1.27.28
Bottleneck	1.3.5
brotlipy	0.7.0
certifi	2022.9.14
cffi	1.15.1
chardet	4.0.0
charset-normalizer	2.0.4
click	8.0.4
cloudpickle	2.0.0
clyent	1.2.2
colorama	0.4.5
colorcet	3.0.0
comtypes	1.1.10
conda	22.9.0
conda-build	3.22.0
conda-content-trust	0.1.3
conda-pack	0.6.0
conda-package-handling	1.9.0
conda-repo-cli	1.0.20
conda-token	0.4.0
conda-verify	3.4.2
constantly	15.1.0
cookiecutter	1.7.3
cryptography	37.0.1
cssselect	1.1.0
cycler	0.11.0
Cython	0.29.32
cytoolz	0.11.0
daal4py	2021.6.0

dask	2022.7.0
datashader	0.14.1
datashape	0.5.4
debugpy	1.5.1
decorator	5.1.1
defusedxml	0.7.1
diff-match-patch	20200713
dill	0.3.4
distributed	2022.7.0
docutils	0.18.1
entrypoints	0.4
et-xmlfile	1.1.0
fastjsonschema	2.16.2
filelock	3.6.0
flake8	4.0.1
Flask	1.1.2
fonttools	4.25.0
fsspec	2022.7.1
future	0.18.2
gensim	4.1.2
glob2	0.7
greenlet	1.1.1
h5py	3.7.0
HeapDict	1.0.1
holoviews	1.15.0
hvplot	0.8.0
hyperlink	21.0.0
idna	3.3
imagecodecs	2021.8.26
imageio	2.19.3
imagesize	1.4.1
importlib-metadata	4.11.3
incremental	21.3.0
inflection	0.5.1
iniconfig	1.1.1
intake	0.6.5
intervaltree	3.1.0
ipykernel	6.15.2
ipython	7.31.1
ipython-genutils	0.2.0
ipywidgets	7.6.5
isort	5.9.3
itemadapter	0.3.0
itemloaders	1.0.4
itsdangerous	2.0.1
jdcal	1.4.1
jedi	0.18.1
jellyfish	0.9.0
Jinja2	2.11.3
jinja2-time	0.2.0
jmespath	0.10.0
joblib	1.1.0
json5	0.9.6
jsonschema	4.16.0
jupyter	1.0.0
jupyter_client	7.3.4
jupyter-console	6.4.3
jupyter_core	4.11.1
jupyter-server	1.18.1
jupyterlab	3.4.4

jupyterlab-pygments	0.1.2
jupyterlab-server	2.10.3
jupyterlab-widgets	1.0.0
keyring	23.4.0
kiwisolver	1.4.2
lazy-object-proxy	1.6.0
libarchive-c	2.9
llvmlite	0.38.0
locket	1.0.0
lxml	4.9.1
lz4	3.1.3
Markdown	3.3.4
MarkupSafe	2.0.1
matplotlib	3.5.2
matplotlib-inline	0.1.6
mccabe	0.6.1
menuinst	1.4.19
mistune	0.8.4
mkl-fft	1.3.1
mkl-random	1.2.2
mkl-service	2.4.0
mock	4.0.3
mpmath	1.2.1
msgpack	1.0.3
multipledispatch	0.6.0
munkres	1.1.4
mypy-extensions	0.4.3
navigator-updater	0.3.0
nbclassic	0.3.5
nbclient	0.5.13
nbconvert	6.4.4
nbformat	5.5.0
nest-asyncio	1.5.5
networkx	2.8.4
nltk	3.7
nose	1.3.7
notebook	6.4.12
numba	0.55.1
numexpr	2.8.3
numpy	1.21.5
numpydoc	1.4.0
olefile	0.46
openpyxl	3.0.10
packaging	21.3
pandas	1.4.4
pandocfilters	1.5.0
panel	0.13.1
param	1.12.0
paramiko	2.8.1
parsel	1.6.0
parso	0.8.3
partd	1.2.0
pathlib	1.0.1
pathspect	0.9.0
patsy	0.5.2
pep8	1.7.1
pexpect	4.8.0
pickleshare	0.7.5
Pillow	9.2.0
pip	22.2.2

pkginfo	1.8.2
platformdirs	2.5.2
plotly	5.9.0
pluggy	1.0.0
poyo	0.5.0
prometheus-client	0.14.1
prompt-toolkit	3.0.20
Protego	0.1.16
psutil	5.9.0
ptyprocess	0.7.0
py	1.11.0
pyasn1	0.4.8
pyasn1-modules	0.2.8
pycodestyle	2.8.0
pycosat	0.6.3
pycparser	2.21
pyct	0.4.8
pycurl	7.45.1
PyDispatcher	2.0.5
pydocstyle	6.1.1
pyerfa	2.0.0
pyflakes	2.4.0
Pygments	2.11.2
PyHamcrest	2.0.2
PyJWT	2.4.0
pylint	2.14.5
pyls-spyder	0.4.0
PyNaCl	1.5.0
pyodbc	4.0.34
pyOpenSSL	22.0.0
pyparsing	3.0.9
pyrsistent	0.18.0
PySocks	1.7.1
pytest	7.1.2
python-dateutil	2.8.2
python-lsp-black	1.0.0
python-lsp-jsonrpc	1.0.0
python-lsp-server	1.3.3
python-slugify	5.0.2
python-snappy	0.6.0
pytz	2022.1
pyviz-comms	2.0.2
PyWavelets	1.3.0
pywin32	302
pywin32-ctypes	0.2.0
pywinpty	2.0.2
PyYAML	6.0
pyzmq	23.2.0
QDarkStyle	3.0.2
qstylizer	0.1.10
QtAwesome	1.0.3
qtconsole	5.2.2
QtPy	2.2.0
queuelib	1.5.0
regex	2022.7.9
requests	2.28.1
requests-file	1.5.1
rope	0.22.0
Rtree	0.9.7
ruamel-yaml-conda	0.15.100

s3transfer	0.6.0
scikit-image	0.19.2
scikit-learn	1.0.2
scikit-learn-intelex	2021.20221004.171935
scipy	1.9.1
Scrapy	2.6.2
seaborn	0.11.2
Send2Trash	1.8.0
service-identity	18.1.0
setuptools	63.4.1
sip	4.19.13
six	1.16.0
smart-open	5.2.1
sniffio	1.2.0
snowballstemmer	2.2.0
sortedcollections	2.1.0
sortedcontainers	2.4.0
soupsieve	2.3.1
Sphinx	5.0.2
sphinxcontrib-applehelp	1.0.2
sphinxcontrib-devhelp	1.0.2
sphinxcontrib-htmlhelp	2.0.0
sphinxcontrib-jsmath	1.0.1
sphinxcontrib-qthelp	1.0.3
sphinxcontrib-serializinghtml	1.1.5
spyder	5.2.2
spyder-kernels	2.2.1
SQLAlchemy	1.4.39
statsmodels	0.13.2
sympy	1.10.1
tables	3.6.1
tabulate	0.8.10
TBB	0.2
tblib	1.7.0
tenacity	8.0.1
terminado	0.13.1
testpath	0.6.0
text-unidecode	1.3
textdistance	4.2.1
threadpoolctl	2.2.0
three-merge	0.1.1
tifffile	2021.7.2
tinycss	0.4
tldextract	3.2.0
toml	0.10.2
tomli	2.0.1
tomlkit	0.11.1
toolz	0.11.2
tornado	6.1
tqdm	4.64.1
traitlets	5.1.1
Twisted	22.2.0
twisted-iocpsupport	1.0.2
typing_extensions	4.3.0
ujson	5.4.0
Unidecode	1.2.0
urllib3	1.26.11
w3lib	1.21.0
watchdog	2.1.6
wcwidth	0.2.5

webencodings	0.5.1
websocket-client	0.58.0
Werkzeug	2.0.3
wheel	0.37.1
widetsnbextension	3.5.2
win-inet-pton	1.1.0
win-unicode-console	0.5
wincertstore	0.2
wrapit	1.14.1
xarray	0.20.1
xlrd	2.0.1
XlsxWriter	3.0.3
xlwings	0.27.15
yapf	0.31.0
zict	2.1.0
zipp	3.8.0
zope.interface	5.4.0

```
In [23]: from sklearn.impute import SimpleImputer

# Create a copy of the raw DataFrame
dfx = dfx_raw.copy()

# Initialize the imputer to replace NaNs with the column mean
imputer = SimpleImputer(missing_values=np.nan, strategy="mean")

# Fit and transform columns at index 1 and 2, then update dfx
dfx.iloc[:, 1:3] = imputer.fit_transform(dfx.iloc[:, 1:3])

## here
```

3. Initialize the Imputer

- **What it does:** Configures the imputation tool with two rules:
 - `missing_values=np.nan` : Look for empty or blank cells (represented as `NaN` in Python).
 - `strategy="mean"` : Calculate the mathematical average of that column to fill those empty cells.

```
In [24]: round(dfx, 2)
```

Out[24]:

	Country	Age	Salary
0	France	44.00	72000.00
1	Spain	27.00	48000.00
2	Germany	30.00	54000.00
3	Spain	38.00	61000.00
4	Germany	40.00	63777.78
5	France	35.00	58000.00
6	Spain	38.78	52000.00
7	France	48.00	79000.00
8	Germany	50.00	83000.00
9	France	37.00	67000.00

In [25]: *# Retriving to the old data set*
 dfx = dfx_raw.copy()

In [26]: **from** sklearn.impute **import** SimpleImputer

Initialize the imputer to replace NaNs with the column constant and take the value w
 imputer = SimpleImputer(missing_values=np.nan, strategy="constant",fill_value = 5)

Fit and transform columns at index 1 and 2, then update dfx
 dfx.iloc[:, 1:3] = imputer.fit_transform(dfx.iloc[:, 1:3])

In [27]: dfx

Out[27]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	5.0
5	France	35.0	58000.0
6	Spain	5.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

for numpy

In [28]: X_raw_copy

```
Out[28]: array([[ 'France', 44.0, 72000.0],
        [ 'Spain', 27.0, 48000.0],
        [ 'Germany', 30.0, 54000.0],
        [ 'Spain', 38.0, 61000.0],
        [ 'Germany', 40.0, nan],
        [ 'France', 35.0, 58000.0],
        [ 'Spain', nan, 52000.0],
        [ 'France', 48.0, 79000.0],
        [ 'Germany', 50.0, 83000.0],
        [ 'France', 37.0, 67000.0]], dtype=object)
```

```
In [29]: X = X_raw_copy
```

```
In [30]: X
```

```
Out[30]: array([[ 'France', 44.0, 72000.0],
        [ 'Spain', 27.0, 48000.0],
        [ 'Germany', 30.0, 54000.0],
        [ 'Spain', 38.0, 61000.0],
        [ 'Germany', 40.0, nan],
        [ 'France', 35.0, 58000.0],
        [ 'Spain', nan, 52000.0],
        [ 'France', 48.0, 79000.0],
        [ 'Germany', 50.0, 83000.0],
        [ 'France', 37.0, 67000.0]], dtype=object)
```

```
In [31]: # Initialize the imputer to replace NaNs with the column constant and take the value w
imputer = SimpleImputer(missing_values=np.nan, strategy="mean")

# Fit and transform columns at index 1 and 2, then update x
X[:, 1:3] = imputer.fit_transform(X[:, 1:3])
```

```
In [32]: print(X)
```

```
[[ 'France' 44.0 72000.0]
 [ 'Spain' 27.0 48000.0]
 [ 'Germany' 30.0 54000.0]
 [ 'Spain' 38.0 61000.0]
 [ 'Germany' 40.0 63777.77777777778]
 [ 'France' 35.0 58000.0]
 [ 'Spain' 38.77777777777778 52000.0]
 [ 'France' 48.0 79000.0]
 [ 'Germany' 50.0 83000.0]
 [ 'France' 37.0 67000.0]]
```

Embedding The Data

```
In [33]: df
```

Out[33]:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

In [36]: `dfx = df.copy()`

In [38]: `dfx = pd.get_dummies(dfx , columns=["Country"])` *#one hot encoding*

In [39]: `dfx`

Out[39]:

	Age	Salary	Purchased	Country_France	Country_Germany	Country_Spain
0	44.0	72000.0	No	1	0	0
1	27.0	48000.0	Yes	0	0	1
2	30.0	54000.0	No	0	1	0
3	38.0	61000.0	No	0	0	1
4	40.0	NaN	Yes	0	1	0
5	35.0	58000.0	Yes	1	0	0
6	NaN	52000.0	No	0	0	1
7	48.0	79000.0	Yes	1	0	0
8	50.0	83000.0	No	0	1	0
9	37.0	67000.0	Yes	1	0	0

In [50]: `dfx = dfx_raw.copy()`
`X = X_raw.copy()`
`Y = y_raw.copy()`
`X = pd.DataFrame(X)`
`Y = pd.DataFrame(Y)`

```
Out[50]:
```

	0	1	2
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [52]: #column Transfer
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Wrap the transformer details in a tuple ( ) inside the transformers list [ ]
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [0])],
    remainder='passthrough'
)

# Fit and transform your feature matrix X
X = np.array(ct.fit_transform(X))

X
```

```
Out[52]: array([[0.0, 1.0, 0.0, 0.0, 44.0, 72000.0],
 [1.0, 0.0, 0.0, 1.0, 27.0, 48000.0],
 [1.0, 0.0, 1.0, 0.0, 30.0, 54000.0],
 [1.0, 0.0, 0.0, 1.0, 38.0, 61000.0],
 [1.0, 0.0, 1.0, 0.0, 40.0, nan],
 [0.0, 1.0, 0.0, 0.0, 35.0, 58000.0],
 [1.0, 0.0, 0.0, 1.0, nan, 52000.0],
 [0.0, 1.0, 0.0, 0.0, 48.0, 79000.0],
 [1.0, 0.0, 1.0, 0.0, 50.0, 83000.0],
 [0.0, 1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

Encoding The Dependent Variable

- leveling for class
- use to convert to the categorical target level into numerical levels
-

```
In [55]: Y
```

Out[55]: **0**

0 No

1 Yes

2 No

3 No

4 Yes

5 Yes

6 No

7 Yes

8 No

9 Yes

```
In [63]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
Y = le.fit_transform(Y)
print(Y)
```

[0 1 0 0 1 1 0 1 0 1]

```
In [69]: le.fit(["India","Paris","paris","tokyo","amsterdam"])
print(le.transform(["India","amsterdam","tokyo","paris"]))
```

[0 2 4 3]

```
In [70]: X
```

```
Out[70]: array([[0.0, 1.0, 0.0, 0.0, 44.0, 72000.0],
 [1.0, 0.0, 0.0, 1.0, 27.0, 48000.0],
 [1.0, 0.0, 1.0, 0.0, 30.0, 54000.0],
 [1.0, 0.0, 0.0, 1.0, 38.0, 61000.0],
 [1.0, 0.0, 1.0, 0.0, 40.0, nan],
 [0.0, 1.0, 0.0, 0.0, 35.0, 58000.0],
 [1.0, 0.0, 0.0, 1.0, nan, 52000.0],
 [0.0, 1.0, 0.0, 0.0, 48.0, 79000.0],
 [1.0, 0.0, 1.0, 0.0, 50.0, 83000.0],
 [0.0, 1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

Splitting The data set for test and train purpose

- we split the data for train and ntest purpsoe
- we use test_size = 0.2 means we split as 80 % to train and 20% to test
- random_state for reproducibility

```
In [75]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,Y,test_size = 0.2,random_state = 1)
```

```
In [76]: X_train
```

```
Out[76]: array([[1.0, 0.0, 0.0, 1.0, nan, 52000.0],
               [1.0, 0.0, 1.0, 0.0, 40.0, nan],
               [0.0, 1.0, 0.0, 0.0, 44.0, 72000.0],
               [1.0, 0.0, 0.0, 1.0, 38.0, 61000.0],
               [1.0, 0.0, 0.0, 1.0, 27.0, 48000.0],
               [0.0, 1.0, 0.0, 0.0, 48.0, 79000.0],
               [1.0, 0.0, 1.0, 0.0, 50.0, 83000.0],
               [0.0, 1.0, 0.0, 0.0, 35.0, 58000.0]], dtype=object)
```

```
In [77]: X_test
```

```
Out[77]: array([[1.0, 0.0, 1.0, 0.0, 30.0, 54000.0],
               [0.0, 1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
In [78]: y_train
```

```
Out[78]: array([0, 1, 0, 0, 1, 1, 0, 1], dtype=int64)
```

```
In [79]: y_test
```

```
Out[79]: array([0, 1], dtype=int64)
```

Feature Scalling

- we apply it on numweical val
- her age and sal are the numerical val

```
In [83]: from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train[:,4:] = sc.fit_transform(X_train[:,4:])
X_test[:, 4:] =sc.transform(X_test[:, 4:])
```

Understanding Z-Scores in Jupyter Notebooks

A **Z-score** (or standard score) measures exactly how many standard deviations a specific data point (X) falls above or below the mean (μ).

The Formula

The mathematical formula to compute a Z-score is:

$$z = \frac{X - \mu}{\sigma}$$

Variable Definitions

- X : The original raw score or value from your dataset.
- μ (**Mu**): The arithmetic mean of the distribution.

- σ (**Sigma**): The standard deviation of the distribution, representing data variability.

In []: